Project - High Level Design

on

Entertainment Content Generator

Course Name: Gen AI

Institution Name: Medicaps University

Course: Datagami Skill Based Course

Student Name(s) & Enrollment Number(s):

Sr no	Student Name	Enrolment Number
1	PRABAL SHUKLA	EN22CS301713
2	PRATIK KOUSHAL	EN22CS301747
3	PRIYANSH CHOUHAN	EN22CS301759
4	PUSHPRAJ SINGH CHOUHAN	EN22CS301774
5	RAJAT MALVIYA	EN22CS301783

Group : 11D7

Project no:GAI-34

Industry Mentor Name: Aashruti Shah

University Mentor Name: Hemlata Patel

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **Entertainment Content Generator** is a Generative AI-based application designed to generate screenplay scenes automatically from user-provided topics. The system uses a combination of **Retrieval-Augmented Generation (RAG)**, vector similarity search, and Large Language Models to produce structured and professional entertainment content.

The application integrates a **React-based frontend**, a **FastAPI backend**, a **FAISS vector database**, and the **Groq API for LLM inference**. The system retrieves domain knowledge related to screenplay formatting and storytelling guidelines before generating the final output.

This approach ensures that generated scenes follow professional screenplay structure including scene headings, character formatting, and dialogue patterns.

1.1 Scope of the Document

This document describes the **High-Level Design (HLD)** of the Entertainment Content Generator system.

The document includes:

- Overall system architecture
- System modules and interactions
- API definitions
- Data design
- Non-functional requirements
- External interfaces

The HLD focuses on the **overall system structure rather than implementation details**.

1.2 Intended Audience

This document is intended for:

- Software architects
- Developers
- System evaluators
- Academic supervisors
- DevOps engineers

The audience should have familiarity with:

- Web application architecture

- REST APIs
- Artificial Intelligence systems
- Cloud-based AI services

1.3 System Overview

The system generates screenplay scenes using a **user-driven input pipeline**.

The system workflow:

1. User enters a topic in the web interface.
2. The frontend sends the request to the backend API.
3. Backend retrieves screenplay guidelines using FAISS vector search.
4. Prompt engineering module constructs a structured prompt.
5. Prompt is sent to the **Groq LLM API**.
6. The LLM generates the screenplay scene.
7. Generated output is returned to the frontend.

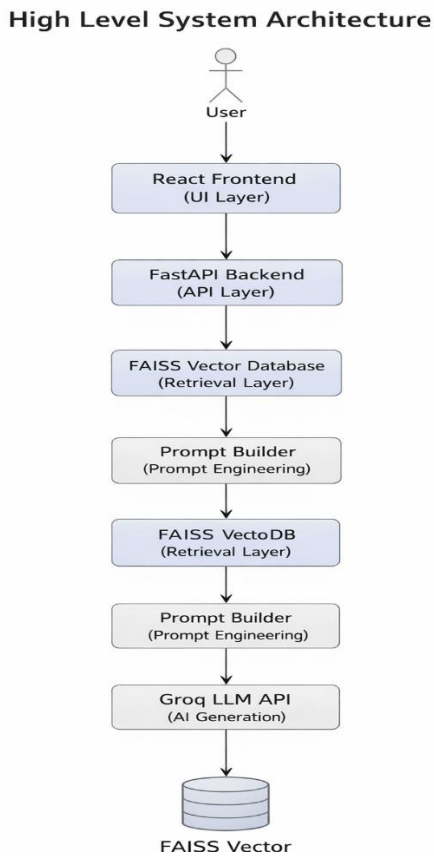

Figure 1: High Level System Architecture Diagram.

2. System Design

The system is designed using a layered architecture separating responsibilities across multiple layers.

Layers include:

- Presentation Layer
- Application Layer
- Retrieval Layer
- AI Processing Layer
- Data Layer

This separation ensures scalability, maintainability, and easier debugging.

2.1 Application Design

The application consists of five major modules.

1 Frontend Layer

Responsibilities:

- User input collection
- Scene generation request
- Output display

Technology: React

2 Backend Layer

Responsibilities:

- API request processing
- Application orchestration
- Communication with retrieval module
- Interaction with LLM API

Technology: FastAPI

3 Retrieval Layer

Responsibilities:

- Query vector database

- Retrieve relevant screenplay knowledge

Technology: FAISS

4 Prompt Engineering Layer

Responsibilities:

- Construct structured prompts
- Combine retrieved context and user input

5 AI Generation Layer

Responsibilities:

- Send prompt to Groq LLM API
- Receive generated screenplay scene

Technology: Groq API

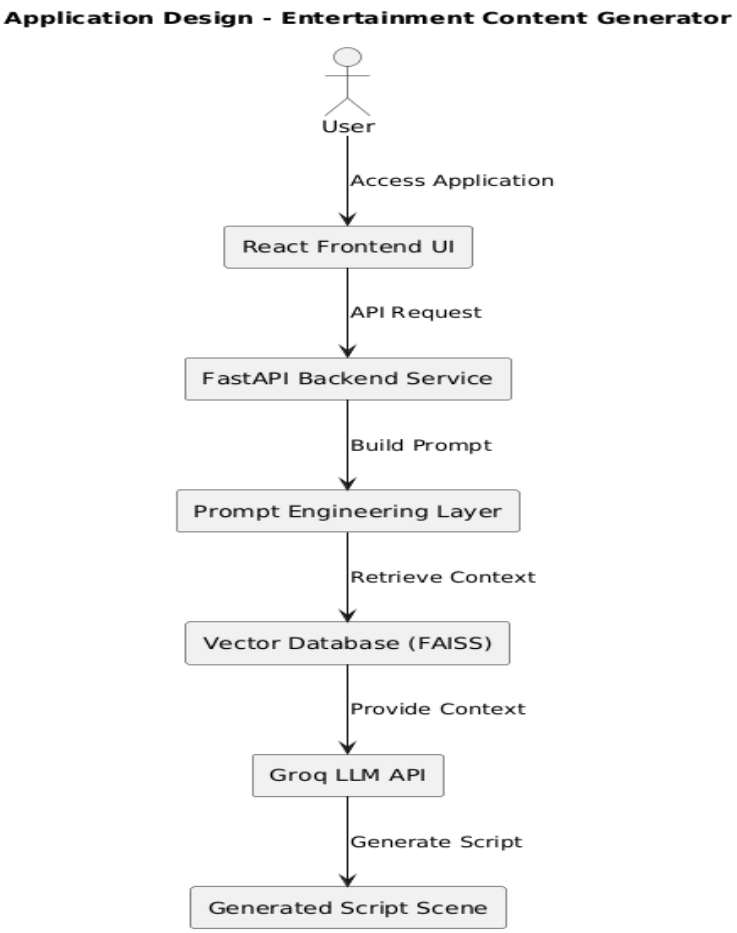

Figure 2: Application Design Diagram.

2.2 Process Flow

The system process flow describes how the system generates screenplay content.

Steps:

1 User enters topic
2 Frontend sends API request
3 Backend receives request
4 Retrieval module searches FAISS database
5 Prompt engine constructs prompt
6 Prompt sent to Groq API
7 LLM generates screenplay scene
8 Backend returns output
9 Frontend displays scene

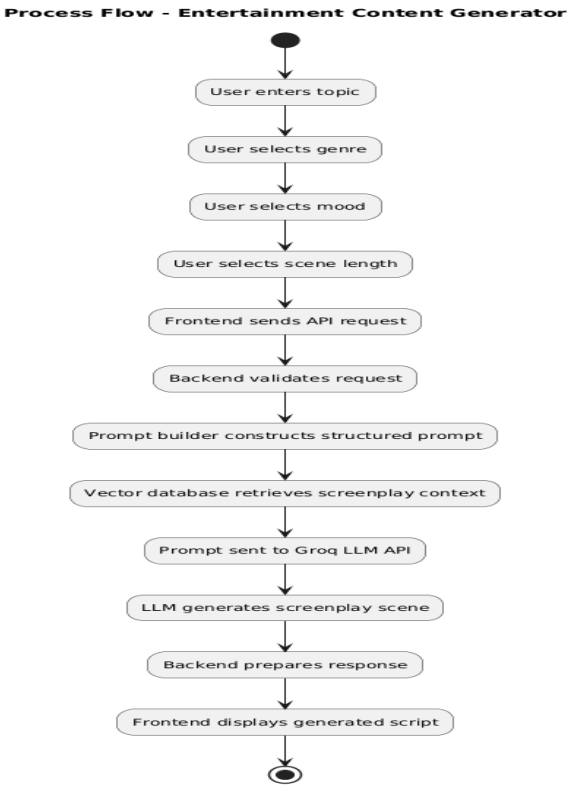

Figure 3: Process Flow Diagram

2.3 Information Flow

Information flow describes how data travels between modules.

Input Data:

- Topic
- Genre
- Mood
- Scene length

Processing Data:

- Context retrieval
- Prompt construction
- AI inference

Output Data:

- Generated screenplay scene
- Retrieved context

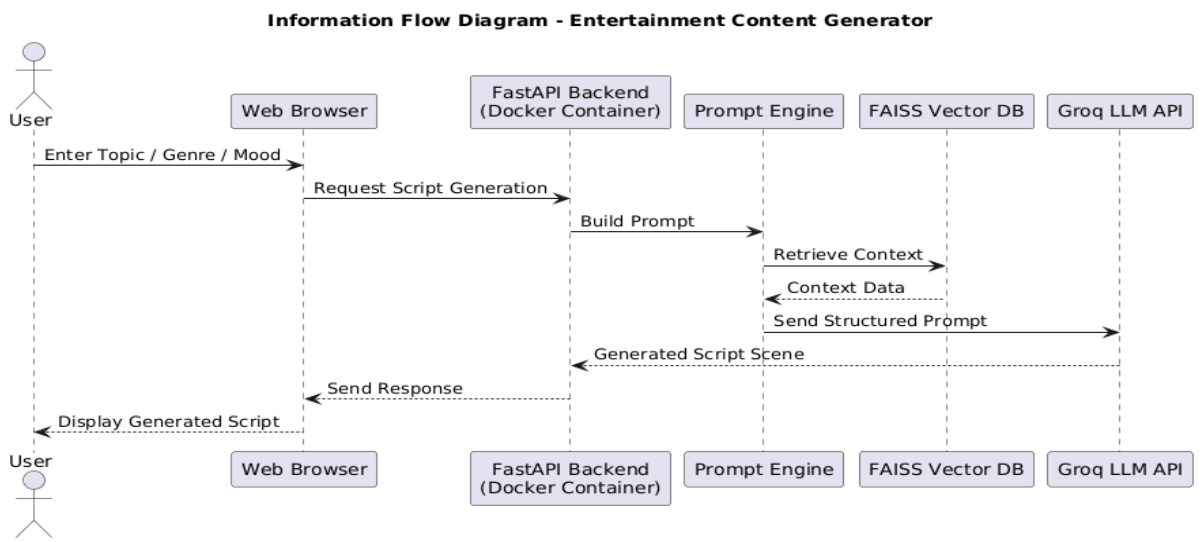

Figure 4: Information Flow Diagram

2.4 Components Design

Major system components:

Frontend

Files:

- App.jsx
- Form.jsx
- Header.jsx
- OutputBox.jsx

Backend

File: app.py

Retriever

File: retriever.py

Prompt Engine

File: prompt_engine.py

Vector Database

Files:

- faiss_index.bin
- docs.pkl

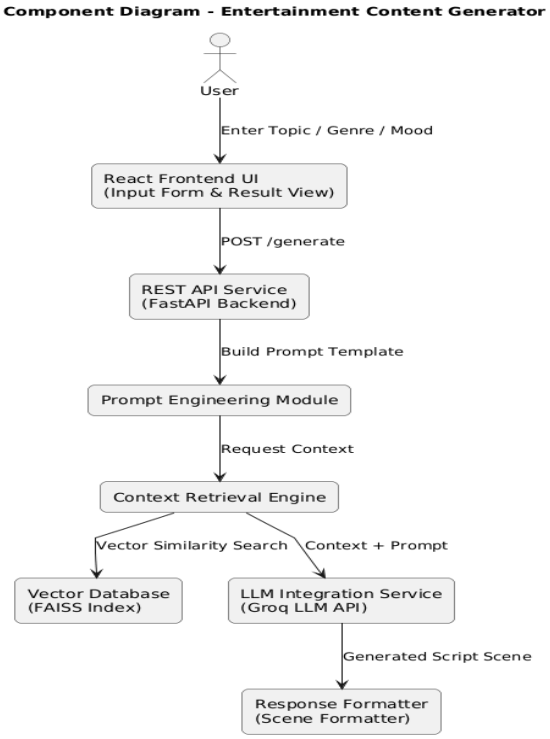

Figure 5: Components Design Diagram

2.5 Key Design Considerations

Scalability
The system can scale horizontally by deploying backend services across multiple instances.

Maintainability
The modular architecture ensures that individual components can be updated independently.

Reliability
The system handles API failures gracefully.

Extensibility
Future features can include:

- character generator
- script summarizer
- multi-scene generation

2.6 API Catalogue

Generate Scene API

Endpoint

POST /generate

```
Request : {  "topic": "string",
             "genre": "string",
             "mood": "string",
             "scene_length": "string" }

Response : { "scene": "generated screenplay",
              "retrieved_context": "text" }
```

3. Data Design

3.1 Data Model

The system stores knowledge documents used for retrieval.

Data includes:

- screenplay guidelines
- dialogue patterns
- industry terminology

3.2 Data Access Mechanism

Data access uses **vector similarity search**.

Steps:

1 Query embedding generation
2 FAISS similarity search
3 Retrieve top matching documents

3.3 Data Retention Policies

The system stores only:

- knowledge base documents
- vector embeddings

Generated scenes are not permanently stored.

3.4 Data Migration

If knowledge base updates occur:

1 New documents added
2 Embeddings recalculated
3 FAISS index rebuilt

4. Interfaces

External interfaces include:

Frontend API Interface
Groq LLM API
Vector database interface

5. State and Session Management

The system follows a **stateless architecture**.

Each request is processed independently.

Benefits:

- improved scalability
- easier load balancing

6. Caching

Caching strategies include:

- prompt caching
- retrieval results caching
- vector embedding caching

This reduces repeated computation.

7. Non-Functional Requirements

7.1 Security Aspects

Security mechanisms include:

- API key protection
- HTTPS communication
- input validation
- rate limiting

7.2 Performance Aspects

Performance improvements include:

- FAISS vector search
- Groq high-speed inference
- asynchronous API calls

8. References

1 React Documentation
2 FastAPI Documentation
3 FAISS Documentation
4 Sentence Transformers Documentation
5 Groq API Documentation